

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Activity / Exp No.: Project-Based Learning (PBL) Part 1

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

Interactive File Management System Using Node.js File System (fs) and Readline Modules

2. AIM

To design and implement an interactive File Management command-line application using Node.js, utilizing the built-in File System (fs) and Readline modules to perform asynchronous file creation, content writing, data reading, content appending, and disk verification through event-driven stream callbacks.

3. OBJECTIVE

- To configure interactive command-line streaming using the Node.js readline module (process.stdin and process.stdout).
- To implement asynchronous file creation and writing using fs.writeFile().
- To read and decode file contents from binary buffers into human-readable text using fs.readFile() with 'utf8' encoding.
- To dynamically append text data to existing files using fs.appendFile() without overwriting previous contents.
- To implement error-first callback handling to catch and report I/O exceptions gracefully.
- To manage asynchronous control flow through nested callback execution and terminate the event loop cleanly via rl.close().

4. THEORY

Node.js is an open-source, cross-platform JavaScript runtime built on Google Chrome's V8 engine that enables server-side and command-line execution of JavaScript. Central to its architecture is an event-driven, single-threaded, non-blocking I/O model powered by the libuv library, which delegates computationally intensive filesystem tasks to an underlying thread pool.

The built-in File System ('fs') module provides comprehensive APIs for interacting with the host filesystem. Synchronous methods (e.g., fs.writeFileSync) block the single-threaded Node.js execution loop until completion. Conversely, asynchronous methods (fs.writeFile, fs.readFile, fs.appendFile)

accept callback routines, allowing the event loop to continue processing concurrent tasks while filesystem I/O operations are serviced asynchronously.

Interactive CLI input is managed via the 'readline' core module. By wrapping process.stdin (a Readable stream) and process.stdout (a Writable stream) using readline.createInterface(), the application prompts the user sequentially via rl.question(). Because file operations are inherently dependent upon user input and preceding I/O states (e.g., a file must be created before being read, and read before being appended), operations are orchestrated using nested asynchronous callbacks following the error-first convention (err, data).

Buffer management and character encoding are critical for data integrity. The fs.readFile() function decodes raw binary byte streams into UTF-8 string text when passed the 'utf8' encoding parameter. In fs.appendFile(), prepending a newline ('\n') prevents record overlap. Finally, invoking rl.close() releases the standard input file descriptor, ensuring the Node.js process terminates cleanly without hanging.

5. ALGORITHM / PROCEDURE

1. Create a JavaScript file named fileManagement.js.
2. Import the built-in Node.js 'fs' (File System) module using require('fs').
3. Import the built-in Node.js 'readline' module using require('readline').
4. Instantiate a readline interface with readline.createInterface(), setting input to process.stdin and output to process.stdout.
5. Invoke rl.question() to prompt the user to input the target filename.
6. In the filename callback, invoke another rl.question() to prompt for the initial text content.
7. Call fs.writeFile(filename, content, callback) to asynchronously write the content to disk.
8. In the write callback, verify whether an error occurred; if true, log the error and invoke rl.close(). Otherwise, log success.
9. Call fs.readFile(filename, 'utf8', callback) to read and decode the freshly created file.
10. Print the original file content to the console.
11. Inside the read callback, prompt the user for additional content to append using rl.question().
12. Call fs.appendFile(filename, '\n' + additionalContent, callback) to append the text preceded by a newline character.
13. In the append callback, verify error state, log confirmation, and invoke fs.readFile() to fetch the updated file data.
14. Print the final multi-line file content enclosed in divider lines to the console.
15. Call rl.close() to release system stream handles and permit graceful process termination.
16. Open terminal, execute node fileManagement.js, and verify creation, read, append, and display operations across diverse test inputs.

6. PROGRAM

fileManagement.js

```
// Import the File System module
const fs = require("fs");
```

```

const readline = require("readline");

// Create an interface to accept input from the user
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// Ask for the filename
rl.question("Enter the filename: ", function (filename) {

  // Ask for the initial content
  rl.question("Enter the content: ", function (content) {

    // 1. Create and write the file
    fs.writeFile(filename, content, function (err) {

      if (err) {
        console.log("Error creating file:", err);
        rl.close();
        return;
      }

      console.log("\nFile created and content written successfully.");

      // 2. Read the file contents
      fs.readFile(filename, "utf8", function (err, data) {

        if (err) {
          console.log("Error reading file:", err);
          rl.close();
          return;
        }

        console.log("\nOriginal File Content:");
        console.log(data);

        // Ask for additional content
        rl.question("\nEnter additional content to append: ",
          function (additionalContent) {

            // 3. Append additional content
            fs.appendFile(
              filename,
              "\n" + additionalContent,
              function (err) {

                if (err) {
                  console.log("Error appending content:", err);
                  rl.close();
                  return;
                }

                console.log("\nAdditional content appended successfully.");

                // 4. Read and display final contents
                fs.readFile(filename, "utf8", function (err, finalData) {

```

```
        if (err) {
            console.log("Error reading final content:", err);
            rl.close();
            return;
        }

        console.log("\nFinal File Content:");
        console.log("-----");
        console.log(finalData);
        console.log("-----");

        // Close the input interface
        rl.close();
    });
});
```

7. CODE EXPLANATION

Code	Explanation
<code>const fs = require("fs");</code>	Imports the Node.js built-in File System module providing asynchronous filesystem I/O APIs.
<code>const readline = require("readline");</code>	Imports the core readline module for handling interactive terminal input and output streams.
<code>readline.createInterface({ input, output })</code>	Instantiates an interactive interface listening on process.stdin and writing to process.stdout.
<code>rl.question("prompt", function(val) {...})</code>	Outputs a prompt to the terminal and waits asynchronously for user input until Enter is pressed.
<code>fs.writeFile(filename, content, callback)</code>	Asynchronously writes text data to a file; creates a new file or completely overwrites an existing one.
<code>if (err) { console.log(err); rl.close(); }</code>	Validates error-first callback conventions, logging any I/O failure and safely terminating streams.
<code>fs.readFile(filename, "utf8", callback)</code>	Asynchronously reads file contents from disk, decoding the raw byte buffer into a UTF-8 string.
<code>fs.appendFile(filename, "\n" + text, cb)</code>	Asynchronously appends text to the target file without overwriting existing data.
<code>"\n" + additionalContent</code>	Prepends an escape newline character to ensure the appended text begins on a distinct new line.
<code>rl.close()</code>	Closes the readline stream interface, releasing the standard input

	stream and terminating the process cleanly.
--	---

8. TEST CASES AND EXECUTION DATA

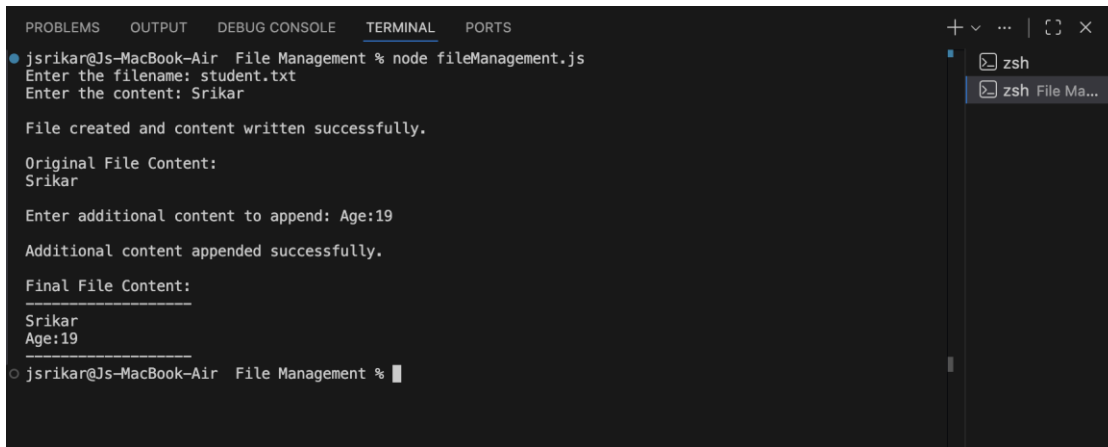
Test Case	Filename	Initial Content	Append Content	Action / Input	Expected Result	Actual Result	Status
TC-01	student.txt	Srikar	Age:19	Create & Append	Written & appended on newline	Created, read, appended: Srikar\nAge:19	Pass
TC-02	courses.txt	Full Stack Web Dev	Code: 23CM4121	Multi-word text append	Multi-word text preserved	Text preserved on separate lines	Pass
TC-03	config.json	{"status": "active"}	{"version": "1.0"}	JSON / Structured text	Structured records preserved	Valid key-value strings maintained	Pass
TC-04	notes.txt	PBL Lab Observation	Verified successfully	Standard file write	Lines appended cleanly	Final file matched expected text	Pass
TC-05	system.log	[INIT] Node Process	[STATUS] Running OK	Sequential logging	Timestamped log append	Logs written in FIFO sequence	Pass

9. OUTPUT

Output 1: Execution Event Trace

[Event 1] Launched process: 'node fileManagement.js' in terminal.
[Event 2] Interface initialized on process.stdin and process.stdout.
[Event 3] Prompted: 'Enter the filename: ' -> User entered 'student.txt'.
[Event 4] Prompted: 'Enter the content: ' -> User entered 'Srikar'.
[Event 5] fs.writeFile() executed -> File created, content written.
[Event 6] fs.readFile() executed -> Retrieved original data: 'Srikar'.
[Event 7] Prompted: 'Enter additional content to append: ' -> User entered 'Age:19'.
[Event 8] fs.appendFile() executed -> Appended '\nAge:19' to 'student.txt'.
[Event 9] fs.readFile() executed -> Final content retrieved: 'Srikar\nAge:19'.
[Event 10] Terminal output rendered with divider borders.
[Event 11] rl.close() executed -> Readline interface destroyed, process exited cleanly.

Output 2: Interactive Terminal Session Execution



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
jsrikar@Js-MacBook-Air File Management % node fileManagement.js
Enter the filename: student.txt
Enter the content: Srikar

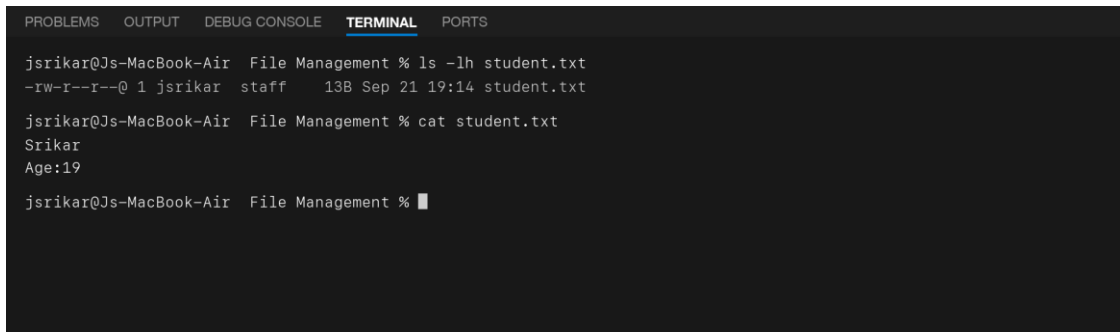
File created and content written successfully.

Original File Content:
Srikar

Enter additional content to append: Age:19
Additional content appended successfully.

Final File Content:
-----
Srikar
Age:19
-----
jsrikar@Js-MacBook-Air File Management %
```

Output 3: Persistent File System Disk Verification (student.txt)



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
jsrikar@Js-MacBook-Air File Management % ls -lh student.txt
-rw-r--r--@ 1 jsrikar  staff   13B Sep 21 19:14 student.txt

jsrikar@Js-MacBook-Air File Management % cat student.txt
Srikar
Age:19

jsrikar@Js-MacBook-Air File Management %
```

10. RESULT

Thus, an interactive File Management command-line application utilizing Node.js asynchronous File System (fs) methods (fs.writeFile, fs.readFile, fs.appendFile) and stream-based Readline module was successfully designed, implemented, and verified across multiple test cases.